Algo Cell INPT

Data Structures & Algorithms with Python

Study Guide

Lists · Stacks · Queues · Linked Lists · Trees · Heaps · Graphs

Contents

1. Module 1: Built-in Data Structures
2. Module 2: Stack
3. Module 3: Queue & Priority Queue
4. Module 4: Linked Lists
5. Module 5: Trees & Binary Search Tree
6. Module 6: Binary Heap
7. Module 7: Graphs
8. Quick Reference Summary

Module 1: Built-in Data Structures

A **data structure** is a way to store and organize data so that it can be accessed effectively.

Built-in	User-defined
1. List	1. Stack
2. Tuple	2. Queue
3. Set	3. Linked List
4. Dictionary	4. Tree
	5. Graph

1.1 List

Creating a list

```
lst  = [1, 2, 3, 4]                 # literal
lst1 = list()                       # empty list via constructor
lst2 = [1, 2, 3, "Sahil", 10.1]     # mixed data types
```

Properties

- **Heterogeneous:** a list can store data of different types.
- **Ordered:** the order of elements matters.

```
list1 = [1, 2, 3, 4, 5]
list2 = [2, 1, 3, 4, 5]
print(list1 == list2)   # False
```

- **Indexed:** items can be accessed with positive or negative indexing.

```
lst = [1, 2, 3, 4, 5]
print(lst[4])    # 5  (positive index)
print(lst[-1])   # 5  (negative index: last element)
```

- **Nestable:** a list can contain other lists.

```
nested = [[1, 2], 1, 2, 3]
```

- **Mutable (changeable):**

```
lst = [1, 2]
lst.append(3)
print(lst)   # [1, 2, 3]
```

- **Dynamic:** lists grow and shrink at runtime.

1.2 Tuple

Creating a tuple

```
tpl = (1, 2, 3, 4)
tpl = tuple()          # empty tuple
single = (5,)          # a single-element tuple needs a trailing comma
```

Properties

- **Immutable (unchangeable):** items cannot be modified after creation.
- Can store data of different types.
- Supports positive and negative indexing.
- Can be nested.

Why do we need tuples?

- A tuple is **faster** than a list.
- Data **cannot be modified**, which protects it from accidental changes.

1.3 Set

A **set** is a collection of **unique** elements.

> **Important:** An empty `{}` creates a **dictionary**, not a set. To create an empty set, use the `set()` constructor.

```
s = set()                 # 1. empty set
s = set("hello")          # 2. {'h', 'e', 'l', 'o'}  (repeated elements are ignored)
s = set([1, 2, 3, 4])     # 3. {1, 2, 3, 4}
s = {1, 2, 3, 4}          # 4. {1, 2, 3, 4}
```

Properties

- **Unordered:** there is no indexing.

```
s  = {1, 2}
s1 = {2, 1}
print(s == s1)   # True
```

- **Mutable:** elements can be added or removed.

```
s.add(3)
```

- Can hold different types of **immutable** objects (numbers, strings, tuples), but not lists, sets, or dicts.

1.4 Dictionary

Syntax

```
dictionary_name = {key1: value1, key2: value2}
```

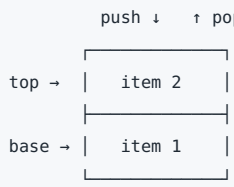
```
d = {}                                      # 1. empty dictionary
d = {"Sahil": "sahil8619@gmail.com"}        # 2. one key-value pair
d = dict()                                  # 3. empty dictionary via constructor
```

Properties

- Keys are **unique**.
- A mutable type (such as a list) **cannot** be used as a key.
- Dictionaries are traditionally described as **unordered**. Since Python 3.7 they preserve insertion order, but they are still not accessed by position.
- Elements **cannot** be accessed using an index.
- Elements are accessed **using the key**: `d["Sahil"]`.
- Dictionaries are **mutable**.
- Dictionaries can be **nested**.

Module 2: Stack

A stack follows **LIFO** (Last In, First Out), also called **FILO** (First In, Last Out).



2.1 Operations

Operation	Description
`push`	Add an element to the top
`pop`	Remove the top element
`peek` / `top`	Fetch the top element without removing it
`isEmpty`	Check whether the stack is empty

2.2 Implementation approaches

1. List
2. Modules (`collections.deque` , `queue.LifoQueue`)

2.3 Using a List

- `push` → `append()`
- `pop` → `pop()`

```
stack = []
stack.append(10)    # [10]
stack.append(20)    # [10, 20]
print(stack.pop())  # 20
print(stack.pop())  # 10
```

Menu-driven stack program

```
stack = []

def push():
    element = input("Enter the element: ")
    stack.append(element)
    print(stack)

def pop_ele():
    if not stack:
        print("Stack is empty!")
    else:
        e = stack.pop()
        print("Removed element:", e)
        print(stack)

while True:
    print("Select the operation: 1. push  2. pop  3. quit")
    choice = int(input())
    if choice == 1:
        push()
    elif choice == 2:
        pop_ele()
    elif choice == 3:
        break
    else:
        print("Enter the correct operation!")
```

2.4 Using Modules

(i) `collections.deque`

```
import collections

stack = collections.deque()   # deque([])
stack.append(10)
stack.append(20)              # deque([10, 20])
print(stack.pop())            # 20
```

(ii) `queue.LifoQueue`

```
import queue

stack = queue.LifoQueue()
stack.put(10)
stack.put(20)
print(stack.get())            # 20
```

> **Note:** `LifoQueue` is thread-safe but not directly printable. Inspect its contents with `stack.queue` (e.g. `[10, 20]`).

Module 3: Queue & Priority Queue

A queue follows **FIFO** (First In, First Out), equivalent to **LILO** (Last In, Last Out).



3.1 Operations

Operation	Description
`enqueue`	Add an element at the rear
`dequeue`	Remove an element from the front
`isFull`	Check whether the queue is full
`isEmpty`	Check whether the queue is empty

3.2 Implementation using a List

- `enqueue` → `append()`
- `dequeue` → `pop(0)`

Approach A: append at the end, pop from the front

```
queue = []
queue.append(10)
queue.append(20)
queue.append(30)
print(queue)        # [10, 20, 30]
print(queue.pop(0)) # 10
print(queue.pop(0)) # 20
```

Approach B: insert at the front, pop from the end

```
queue = []
queue.insert(0, 10)
queue.insert(0, 20)
queue.insert(0, 30)
print(queue)        # [30, 20, 10]
print(queue.pop())  # 10
print(queue.pop())  # 20
```

Checking for empty

```
queue = []
print(not queue)    # True  (queue is empty)
```

Menu-driven queue program

```
queue = []

def enqueue():
    element = input("Enter the element: ")
    queue.append(element)
    print(element, "is added to queue!")

def dequeue():
    if not queue:
        print("Queue is empty!")
    else:
        e = queue.pop(0)
        print("Removed element:", e)

def display():
    print(queue)

while True:
    print("Select the operation: 1. add  2. remove  3. show  4. quit")
    choice = int(input())
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        display()
    elif choice == 4:
        break
    else:
        print("Enter the correct operation!")
```

Complexity note: `list.pop(0)` is O(n) because every remaining element shifts left. `collections.deque` gives O(1) at both ends.

3.3 Implementation using Collections: `deque`




```
import collections

q = collections.deque()       # deque([])

# Variant 1: appendleft + pop
q.appendleft(10)
q.appendleft(20)              # deque([20, 10])
print(q.pop())                # 10
print(q.pop())                # 20

# Variant 2: append + popleft
q.append(10)
q.append(20)                  # deque([10, 20])
print(q.popleft())            # 10
print(q.popleft())            # 20

print(not q)                  # True (empty)
```

3.4 Implementation using `queue.Queue`

```
import queue

q = queue.Queue()
q.put(10)
q.put(20)
q.put(30)
print(q.get())   # 10
print(q.get())   # 20
print(q.get())   # 30
```

3.5 Priority Queue

Elements are served by priority rather than arrival order. Two conventions exist:

1. **Low → High priority:** the smallest value is served first (min-priority).
2. **High → High priority:** the largest value is served first (max-priority).

(i) Using a List

```
q = []
q.append(10)
q.append(20)
q.append(40)
q.sort()               # [10, 20, 40]
print(q.pop(0))        # 10  (smallest first)
print(q.pop(0))        # 20
```

> **Note (correction):** The handwritten notes used `q.pop()`, which removes the *last* element (40) from a sorted list. To serve the smallest value first, use `pop(0)`. For the "high → high" convention, `pop()` on the sorted list is correct.

(ii) Using the `PriorityQueue` class

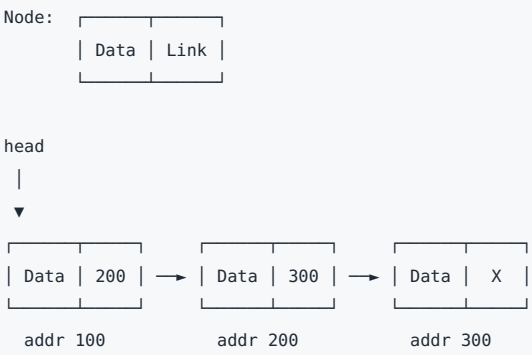
```
import queue

q = queue.PriorityQueue()
q.put(10)
q.put(60)
q.put(20)
q.put(40)
print(q.get())   # 10
print(q.get())   # 20
print(q.get())   # 40
print(q.get())   # 60
```

`PriorityQueue` is backed by a min-heap, so the smallest value always comes out first.

Module 4: Linked Lists

A linked list is a chain of **nodes**. Each node stores **data** and a **link (reference)** to the next node.



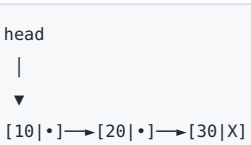
Key points

- **Node:** an element of the linked list.
- A **dynamic** data structure.
- **Insertion and deletion are easy** (no shifting of elements).
- Used to implement **stacks, queues, and graphs**.
- Used to **represent and manipulate polynomials**.
- Needs **extra memory** for the link field.
- **Random access is not possible**; you must traverse from the head.

Types

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

4.1 Singly Linked List (SLL)



Operations

1. **Add** (at beginning, at end, in between)
2. **Delete** (at beginning, at end, in between / by value)
3. **Traversal**

Node and LinkedList classes (base for all SLL operations)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.ref = None          # reference to the next node


class LinkedList:
    def __init__(self):
        self.head = None

    # ---------- Traversal ----------
    def print_ll(self):
        if self.head is None:
            print("Linked list is empty!")
        else:
            n = self.head
            while n is not None:
                print(n.data, "-->", end=" ")
                n = n.ref
            print()
```

1. Insertion

(i) Add at the beginning

Steps:

1. Create the new node.
2. `new_node.ref = head`
3. `head = new_node`

```
new_node ──► [ ]──►[ ]──►[ ]──►[ ]
  ▲
head
```

```
    def add_begin(self, data):
        new_node = Node(data)
        new_node.ref = self.head
        self.head = new_node
```

(ii) Add at the end

Steps:

1. Create the new node.
2. Go to the last node `n`.
3. `n.ref = new_node`

```
head ─►[ ]─►[ ]─►[ ]─►[ ]
                      |
                      └─► [new_node]
```

```
    def add_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            n = self.head
            while n.ref is not None:
                n = n.ref
            n.ref = new_node
```

(iii) Add in between

(A) Insert after a given node `x`

```
head ─►[ ]─►[n]─ X ─►[ ]─►[X]
              │       ▲
              └►[new]─┘
```

```
    def add_after(self, data, x):
        n = self.head
        while n is not None:
            if x == n.data:
                break
            n = n.ref
        if n is None:
            print("Node is not present")
        else:
            new_node = Node(data)
            new_node.ref = n.ref
            n.ref = new_node
```

(B) Insert before a given node `x`

```
    def add_before(self, data, x):
        if self.head is None:
            print("Empty")
            return
        # Special case: x is the head node
        if self.head.data == x:
            new_node = Node(data)
            new_node.ref = self.head
            self.head = new_node
            return
        n = self.head
        while n.ref is not None:
            if n.ref.data == x:
                break
            n = n.ref
        if n.ref is None:
            print("Node is not found!")
        else:
            new_node = Node(data)
            new_node.ref = n.ref
            n.ref = new_node
```

(C) Insert into an empty linked list

```
    def insert_empty(self, data):
        if self.head is None:
            new_node = Node(data)
            self.head = new_node
        else:
            print("Linked list is not empty!")
```

Usage:

```
ll1 = LinkedList()
ll1.insert_empty(10)
ll1.add_begin(5)
ll1.add_end(20)
ll1.add_after(15, 10)
ll1.add_before(1, 5)
ll1.print_ll()      # 1 --> 5 --> 10 --> 15 --> 20 -->
```

2. Deletion

(i) Delete the first node

```
    def delete_begin(self):
        if self.head is None:
            print("Linked list is empty!")
        else:
            self.head = self.head.ref
```

(ii) Delete the last node

```
    def delete_end(self):
        if self.head is None:
            print("Linked list is empty!")
        elif self.head.ref is None:      # only one node
            self.head = None
        else:
            n = self.head
            while n.ref.ref is not None:
                n = n.ref
            n.ref = None
```

(iii) Delete a node in the middle / by value

```
n = 20
head ─►[10|•]── X ─►[20|•]── X ─►[30|•]
              └──────────────────►┘
```

```
    def delete_by_value(self, x):
        if self.head is None:
            print("Linked list is empty!")
            return
        if x == self.head.data:
            self.head = self.head.ref
            return
        n = self.head
        while n.ref is not None:
            if x == n.ref.data:
                break
            n = n.ref
        if n.ref is None:
            print("Node is not present!")
        else:
            n.ref = n.ref.ref
```

4.2 Doubly Linked List (DLL)

Each node has two references: `pref` (previous) and `nref` (next).

```
        head
         |
         ▼
 X ◄─[ | |•]◄──►[•| |•]◄──►[•| |•]─► X
```

Operations

- **Insertion:** begin, end, middle
- **Deletion:** begin, end, middle / by value
- **Traversal:** forward, backward

Node and DoublyLL classes

```
class Node:
    def __init__(self, data):
        self.data = data
        self.nref = None     # next reference
        self.pref = None     # previous reference


class DoublyLL:
    def __init__(self):
        self.head = None
```

1. Traversal

```
    def print_ll(self):                     # forward traversal
        if self.head is None:
            print("Linked list is empty!")
        else:
            n = self.head
            while n is not None:
                print(n.data, "-->", end=" ")
                n = n.nref
            print()

    def print_ll_reverse(self):             # backward traversal
        if self.head is None:
            print("Linked list is empty!")
        else:
            n = self.head
            while n.nref is not None:       # go to the last node
                n = n.nref
            while n is not None:
                print(n.data, "-->", end=" ")
                n = n.pref
            print()
```

2. Insertion

Insertion types: when empty, at beginning, at end, after a node, before a node.

```
    def insert_empty(self, data):
        if self.head is None:
            new_node = Node(data)
            self.head = new_node
        else:
            print("Linked list is not empty!")

    def add_begin(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            new_node.nref = self.head
            self.head.pref = new_node
            self.head = new_node

    def add_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            n = self.head
            while n.nref is not None:
                n = n.nref
            n.nref = new_node
            new_node.pref = n

    def add_after(self, data, x):
        if self.head is None:
            print("Linked list is empty!")
        else:
            n = self.head
            while n is not None:
                if x == n.data:
```

```
                    break
                n = n.nref
            if n is None:
                print("Given node is not present in LL")
            else:
                new_node = Node(data)
                new_node.nref = n.nref
                new_node.pref = n
                if n.nref is not None:
                    n.nref.pref = new_node
                n.nref = new_node

    def add_before(self, data, x):
        if self.head is None:
            print("LL is empty!")
        else:
            n = self.head
            while n is not None:
                if x == n.data:
                    break
                n = n.nref
            if n is None:
                print("Node not present in LL")
            else:
                new_node = Node(data)
                new_node.nref = n
                new_node.pref = n.pref
                if n.pref is not None:
                    n.pref.nref = new_node
                else:
                    self.head = new_node     # inserting before the head
                n.pref = new_node
```

Usage:

```
dl1 = DoublyLL()
dl1.add_begin(4)
dl1.add_after(10, 4)
dl1.add_before(20, 10)
dl1.add_end(30)
dl1.print_ll()     # 4 --> 20 --> 10 --> 30 -->
```

3. Deletion

Deletion types: beginning, end, by value.

```
    def delete_begin(self):
        if self.head is None:
            print("DLL is empty!")
            return
        if self.head.nref is None:           # only one node
            self.head = None
            print("DLL is empty after deleting node!")
        else:
            self.head = self.head.nref
            self.head.pref = None

    def delete_end(self):
        if self.head is None:
            print("DLL is empty!")
            return
        if self.head.nref is None:           # only one node
            self.head = None
            print("DLL is empty after deleting the node")
        else:
            n = self.head
            while n.nref is not None:
                n = n.nref
            n.pref.nref = None

    def delete_by_value(self, x):
        if self.head is None:
            print("DLL is empty!")
            return
        # Only one node
        if self.head.nref is None:
            if x == self.head.data:
                self.head = None
            else:
                print("X is not present")
            return
        # Head holds the value
        if self.head.data == x:
            self.head = self.head.nref
            self.head.pref = None
            return
        # Search the rest of the list
        n = self.head
        while n.nref is not None:
            if x == n.data:
                break
            n = n.nref
        if n.nref is not None:               # middle node
            n.nref.pref = n.pref
            n.pref.nref = n.nref
        else:                                # last node
            if n.data == x:
                n.pref.nref = None
            else:
                print("X is not present")
```

Usage:

```
dl1 = DoublyLL()
dl1.add_begin(4)
dl1.add_before(10, 4)
dl1.delete_end()
dl1.add_begin(10)
dl1.add_begin(20)
dl1.delete_begin()
dl1.delete_by_value(10)
dl1.print_ll()
```

Module 5: Trees & Binary Search Tree

5.1 Tree basics

- A **non-linear** data structure.
- Represents **relationships between nodes**.
- A collection of entities called **nodes**.
- Nodes are connected by **edges**.

```
              (Root)
             /      \
        (Child)       (Parent)
        /    \        /      \
    (leaf) (leaf) (leaf)  (Internal node: at least one child)
                             /   |   \
                         (leaf)(leaf)(leaf)
```

Terms: Root, Edge, Parent, Child, Siblings (same parent), Internal node (at least one child), Leaf node (no children).

5.2 Characteristics

```
            (A)                 ---- Level 0
           /   \
       (B)     (C)              ---- Level 1
      /  \     /  \
    (d)  (e) (f)   (g)          ---- Level 2
          |       / | \  \
         (h)    (i)(j)(k)       ---- Level 3
```

(In the original diagram, B has children d and e, e has child h, and g has children i, j, k.)

1. **Root:** the top-most node.
2. A tree with **N nodes has N − 1 edges (links).**
3. Every child has **only one parent**.
4. A tree is a **recursive** data structure.
5. **Degree of a node:** total number of children of that node.
6. **Degree of a tree:** the highest degree among its nodes.
7. **Level:** each step from top to bottom. The root is at level 0.
8. **Height of a node:** the number of edges on the longest path from that node down to a leaf. Examples: C → 2, B → 2, A → 3.
9. **Height of a tree:** the height of the root (A → 3).
10. **Depth of a node:** the number of edges from the root to that node (C → 1).
11. **Depth of a tree:** the longest path from the root to a leaf.

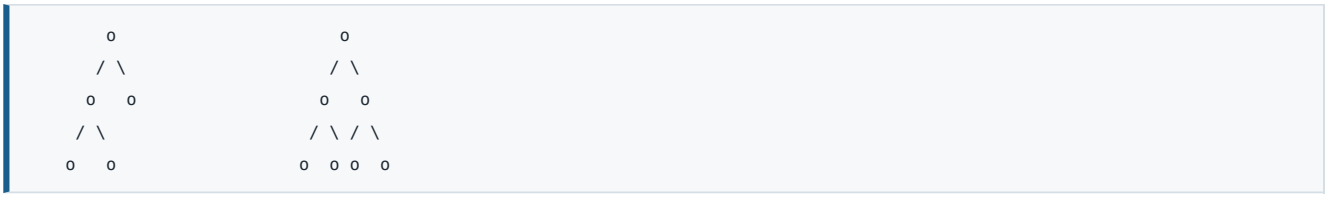
5.3 Binary Tree

Each node can have **0, 1, or 2** children.

Types: Full, Complete, Perfect, Balanced, Pathological (Degenerate)

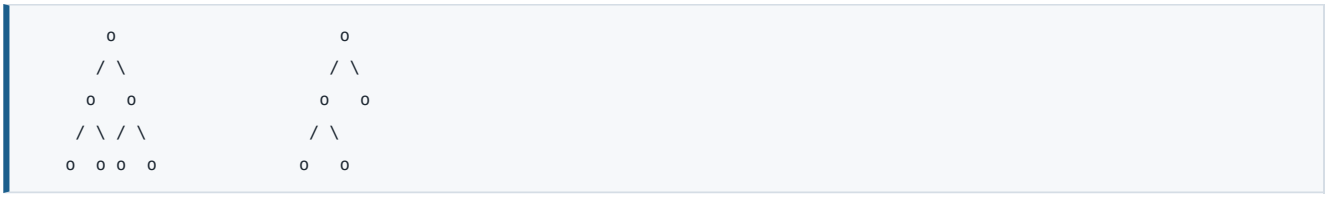
1. Full Binary Tree

Every node has **0 or 2** children. Every node other than the leaves has exactly 2 children.



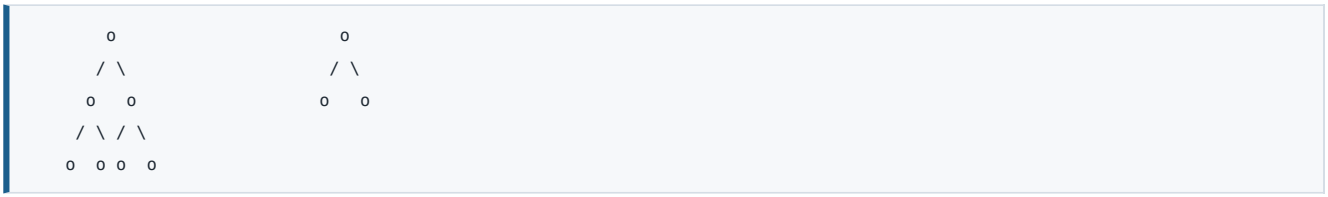
2. Complete Binary Tree

All levels are completely filled, except possibly the last level, which is filled **from left to right**.



3. Perfect Binary Tree

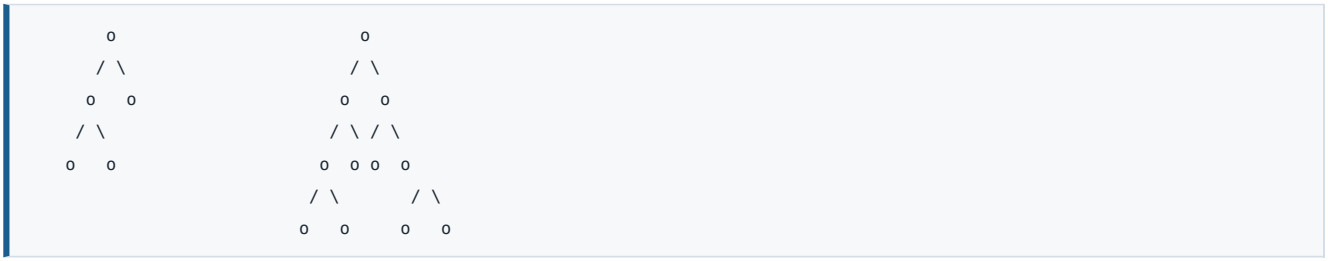
All internal nodes have **exactly two children**, and all leaf nodes are at the **same level**.



> **Note (correction):** The handwritten text added "all nodes in the last level are as far left as possible". That condition belongs to the *Complete* binary tree. A perfect tree has a completely filled last level, so it is automatically left-aligned.

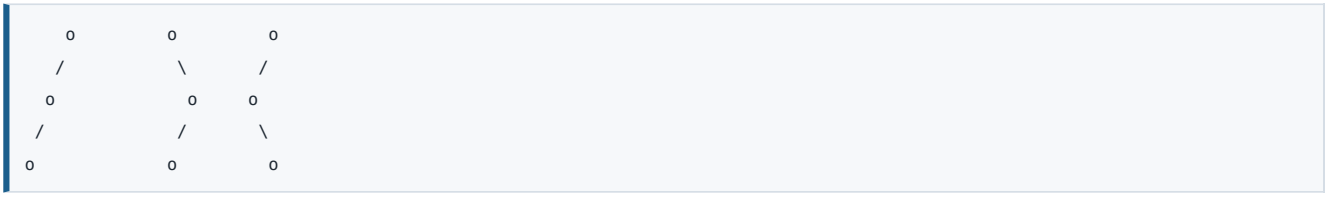
4. Balanced Binary Tree

The height difference between the left and right subtrees of **any node is at most one**. Common balanced trees include **AVL trees, Red-Black trees, and B-trees.**



5. Pathological (Degenerate) Binary Tree

Each parent node has **only one child or no child**. The tree is essentially a linear data structure, resembling a linked list rather than a tree.



5.4 Binary Search Tree (BST)

- The **left subtree** of a node contains only nodes with keys **less than** the node's key.
- The **right subtree** of a node contains only nodes with keys **greater than** the node's key.

- The left and right subtrees must each also be a BST.

```
            (100)
           /     \
        (50)     (200)
        /  \     /   \
     (30) (70) (150) (400)
```

Duplicate handling conventions:

- Duplicate values **not allowed** (the version implemented here).
- `Left <= root < right`
- `Left < root <= right`

BST operations (concepts)

1. Search

1. Check whether the BST is empty. If yes, print a message. If no, compare the root key with the given value.
2. If root == value, print "Key found". Otherwise decide which side to search.
3. If value < root key, search the left subtree. Otherwise search the right subtree. Repeat from step 1 on that subtree.

2. Insertion

1. Check whether the BST is empty. If yes, insert the new node as the root.
2. Otherwise compare the root key with the given value.
3. If root key < value, go to the right subtree. Otherwise go to the left subtree. Find the correct position and insert.

3. Deletion

1. **Leaf node:** simply remove it.
2. **One child:** replace the node with its child.
3. **Two children:** find the node with the minimum key in its right subtree (the in-order successor), replace the node's key with it, then delete that successor from its original position. (The maximum key of the left subtree also works.)

4. Traversal

Traversal	Order
Pre-order	Visit root → traverse left subtree → traverse right subtree
In-order	Traverse left subtree → visit root → traverse right subtree
Post-order	Traverse left subtree → traverse right subtree → visit root

> In-order traversal of a BST yields keys in **sorted ascending order**.

5.5 BST Implementation in Python

```
       [ Data ]
      /        \
left pointer   right pointer
```

Node class (with insertion)

```
class BST:
    def __init__(self, key):
        self.key = key
        self.lchild = None
        self.rchild = None

    # ---------- 1. Insertion ----------
    def insert(self, data):
        if self.key is None:
            self.key = data
            return
        if self.key == data:          # duplicates are ignored
            return
        if self.key > data:
            if self.lchild:
                self.lchild.insert(data)
            else:
                self.lchild = BST(data)
        else:
            if self.rchild:
                self.rchild.insert(data)
            else:
                self.rchild = BST(data)
```

Note (correction): In the handwritten code, the duplicate check (`if self.key == data: return`) appeared at the end of the method, where it could never prevent a duplicate insertion. It must come *before* the left/right branching.

```
root = BST(10)
for i in [20, 4, 30, 4, 1, 5, 6]:
    root.insert(i)
```

Resulting tree (the duplicate `4` is ignored):

```
           10
         /    \
        4      20
       / \       \
      1   5       30
           \
            6
```

2. Search

```
    def search(self, data):
        if self.key == data:
            print("Node is found!")
            return
        if data < self.key:
            if self.lchild:
                self.lchild.search(data)
            else:
                print("Node is not present!")
        else:
            if self.rchild:
                self.rchild.search(data)
            else:
                print("Node is not present!")
```

```
root = BST(10)
for i in [6, 3, 1, 98, 3, 7]:
    root.insert(i)
root.search(7)      # Node is found!
```

3. Traversal

```
    def preorder(self):                 # Root → Left → Right
        print(self.key, end=" ")
        if self.lchild:
            self.lchild.preorder()
        if self.rchild:
            self.rchild.preorder()

    def inorder(self):                  # Left → Root → Right
        if self.lchild:
            self.lchild.inorder()
        print(self.key, end=" ")
        if self.rchild:
            self.rchild.inorder()

    def postorder(self):                # Left → Right → Root
        if self.lchild:
            self.lchild.postorder()
        if self.rchild:
            self.rchild.postorder()
        print(self.key, end=" ")
```

For the tree built from `[10, 20, 4, 30, 4, 1, 5, 6]`:

Traversal	Output
Pre-order	`10 4 1 5 6 20 30`
In-order	`1 4 5 6 10 20 30`
Post-order	`1 6 5 4 30 20 10`

4. Deletion

```
    def delete(self, data):
        if self.key is None:
            print("Tree is empty!")
            return self
        if data < self.key:
            if self.lchild:
                self.lchild = self.lchild.delete(data)
            else:
                print("Given node is not present")
        elif data > self.key:
            if self.rchild:
                self.rchild = self.rchild.delete(data)
            else:
                print("Given node is not present")
        else:
            # Case 1 & 2: zero or one child
            if self.lchild is None:
                return self.rchild
            if self.rchild is None:
                return self.lchild
            # Case 3: two children, find the in-order successor
            node = self.rchild
            while node.lchild:
                node = node.lchild
            self.key = node.key                                 # copy successor's key
            self.rchild = self.rchild.delete(node.key)          # delete successor
        return self
```

Note (correction): The handwritten version used `self = None` before returning, which has no effect on the parent's reference. Returning the replacement child (or `None` for a leaf) lets the parent re-link properly. `return self` also had to apply to every branch, not just the last.

5. Min and Max node

```
    def min_node(self):
        current = self
        while current.lchild:
            current = current.lchild
        print("Smallest key is:", current.key)

    def max_node(self):
        current = self
        while current.rchild:
            current = current.rchild
        print("Maximum key is:", current.key)
```

Module 6: Binary Heap

A binary heap is a **complete binary tree** that satisfies the heap property.

Type	Property
Min Heap	The key of every parent node is **less than or equal to** its children's keys
Max Heap	The key of every parent node is **greater than or equal to** its children's keys

6.1 Operations

1. Heapify

A process that rearranges the elements of the heap to **maintain the heap property**.

- Makes sure every node of the tree follows the heap property.
- Used to create a binary heap from a complete binary tree.

Where heapify is used:

- Insertion
- Deletion
- Creating a binary heap from a given array

Two types of heapify:

(1) `heapify_up`**.** Performed when we **insert** a new node. The new node moves up toward the root until the property holds. *Also known as:* up-heap, bubble-up, percolate-up, shift-up, trickle-up, swim-up, cascade-up.

(2) `heapify_down`**.** Performed when we **delete** a node. The replacement node moves down toward the leaves until the property holds. *Also known as:* down-heap, bubble-down, percolate-down, sift-down, extract min/max, sink-down.

```
heapify_up:   leaf ↗ ... ↗ root
heapify_down: root ↘ ... ↘ leaf
```

2. Insertion

Insert a new node while maintaining the heap properties.

1. Add the new node to the **first open spot** in the lowest level (keeps the tree complete).
2. **Heapify (up)** the new node.

3. Deletion

Remove a node while maintaining the heap properties.

1. **Swap** the node you want to delete with the **last node**.
2. **Delete** the last node.
3. **Heapify (down)** the node that now sits in the deleted node's position.

6.2 Construction of a Binary Heap

Steps:

1. First create a complete binary tree from the given list of numbers.
2. Then start heapifying the tree, **starting from the last internal node** and moving back to the root.

Example (build a Max-Heap): Numbers: `[5, 1, 4, 10, 20]`

```
Step 1: Complete binary tree       Step 2: Heapify node "1"
                                   (swap 1 with larger child 20)
        5                                  5
       / \                                / \
      1   4                             20   4
     / \                                / \
   10   20                            10   1

Step 3: Heapify root "5"           Step 4: Continue sinking "5"
(swap 5 with larger child 20)      (swap 5 with larger child 10)
       20                                 20
      /  \                               /  \
     5    4                            10    4
    / \                                / \
  10   1                             5   1          <- Max-Heap
```

Final array: `[20, 10, 4, 5, 1]`

6.3 Array Representation

```
               100
             /     \
           50       70
          /  \     /  \
        10   30   40   50
       /  \
      1    7
```

Index	0	1	2	3	4	5	6	7	8
Value	100	50	70	10	30	40	50	1	7

Index formulas (0-based):

Relationship	Formula
Root	`list[0]`
i-th node	`list[i]`
Parent of i	`(i - 1) // 2`
Left child of i	`(2 * i) + 1`
Right child of i	`(2 * i) + 2`

6.4 Implementation with `heapq`

Python's `heapq` module implements a **min-heap** on top of a plain list.

```
import heapq

heap = []
heapq.heappush(heap, 10)
heapq.heappush(heap, 1)
heapq.heappush(heap, 5)
print(heap)                  # [1, 10, 5]
```

```
    1
   / \
  10   5
```

```
heapq.heappop(heap)          # removes the smallest value → 1
heapq.heappop(heap)          # next smallest → 5
```

Other `heapq` functions

```
list1 = [1, 3, 5, 2, 4, 6]

heapq.heapify(list1)                # in-place heapify
print(list1)                        # [1, 2, 5, 3, 4, 6]

heapq.heappushpop(list1, 89)        # push 89, then pop smallest (returns 1)
print(list1)                        # [2, 3, 5, 89, 4, 6]

heapq.heapreplace(list1, 100)       # pop smallest (returns 2), then push 100
print(list1)                        # [3, 4, 5, 89, 100, 6]

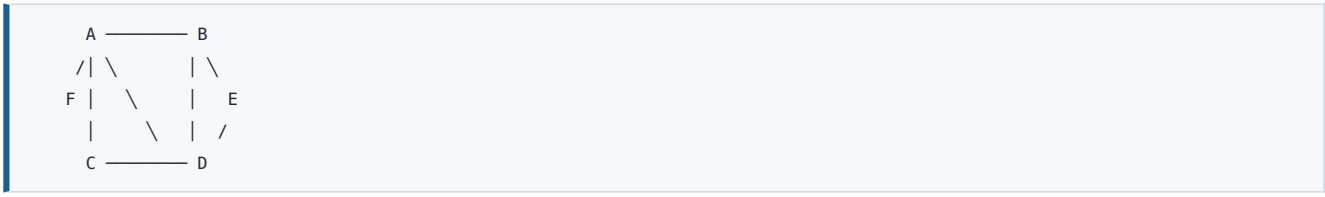
data = [1, 20, 5, 4, 3, 6, 2]
print(heapq.nsmallest(2, data))     # [1, 2]
print(heapq.nlargest(3, data))      # [20, 6, 5]
```

Function	Behavior
`heappush(h, x)`	Push `x`, keeping the heap property
`heappop(h)`	Pop and return the smallest item
`heapify(list)`	Convert a list to a heap in place, O(n)
`heappushpop(h, x)`	Push then pop (more efficient than separate calls)
`heapreplace(h, x)`	Pop then push
`nsmallest(n, it)` / `nlargest(n, it)`	Return the n smallest or largest items from any iterable

Complexity: push O(log n), pop O(log n), heapify O(n), peek (`h[0]`) O(1).

Module 7: Graphs

A **graph** is a non-linear data structure consisting of **nodes (vertices)** and **edges**.



Formal definition: `G = (V, E)`

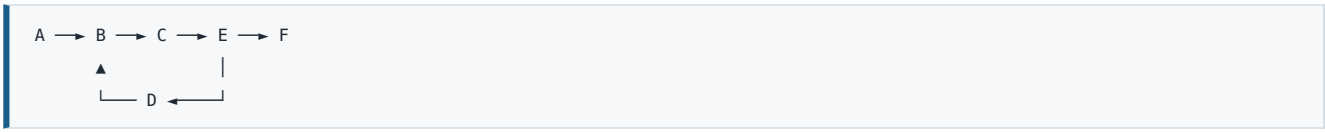
- `G` = Graph
- `V` = set of vertices
- `E` = set of edges
- `( )` = ordered pair (directed edge)
- `{ }` = unordered pair (undirected edge)

Uses: Google Maps and GPS, Facebook and LinkedIn, E-commerce websites.

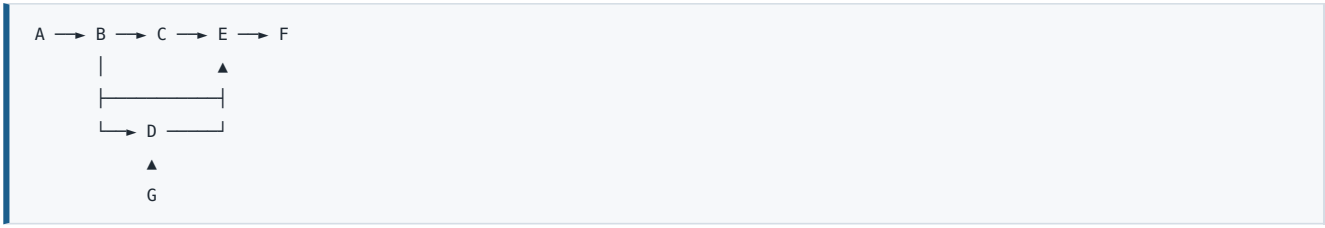
7.1 Types of Graphs

#	Type	Description	Diagram
1	**Directed**	All edges are unidirectional	`(A) ⟶ (B)`
2	**Undirected**	All edges are bidirectional	`(A) —— (B)`
3	**Weighted**	Each edge has a weight/cost/value	`(A) –50km– (B)`
4	**Unweighted**	No value or weight on edges	`(A) —— (B)`
5	**Cyclic**	Contains at least one cycle	see below
6	**Acyclic**	Contains no cycles	see below

Cyclic graph (cycle: B → C → E → D → B):



Acyclic graph (a directed acyclic graph, or DAG):



(Edges: A→B, B→C, B→E, B→D, C→E, D→E, G→D, E→F.)

7.2 Terminology

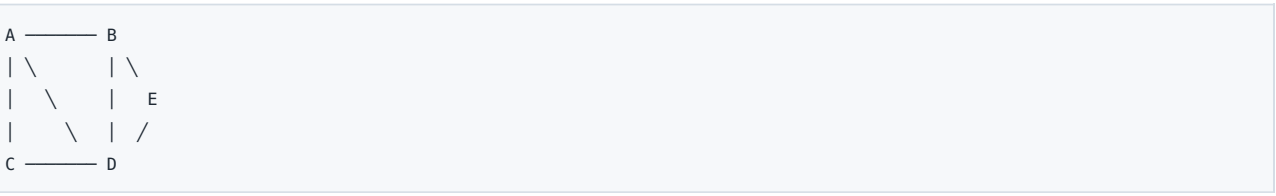
1. **Adjacent (neighbour) nodes:** node X is adjacent to node Y if there is an edge between X and Y.
2. **Path:** a sequence of vertices in which each pair of successive nodes is connected by an edge. Example: `A → G` via `A → C → F → G`.
3. **Cycle:** a path in which the first and last node are the same and all other nodes are distinct.
4. **Connected graph:** there is a path from any node to any other node.
5. **Degree:**
    - **Degree** of a node = number of edges connected to it.

- **Indegree** = number of edges coming *into* that node.
- **Outdegree** = number of edges going *out* from that node.

6. **Complete graph:** a simple undirected graph in which every pair of vertices is adjacent.

7.3 Graph Representation

Consider this undirected graph:



Edges: A–B, A–C, A–D, B–D, B–E, C–D, D–E.

1. Adjacency Matrix

Represents connections between nodes in matrix form.

Steps:

1. Create a `K × K` matrix (K = number of nodes).
2. Rows and columns represent the nodes of the graph.
3. If an edge is present store `1`, otherwise store `0`.

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	0	1
E	0	1	0	1	0

2. Adjacency List

An array (or dictionary) of lists, where each element represents a vertex and its list contains the vertices adjacent to it.

```
A: [B, C, D]
B: [A, D, E]
C: [A, D]
D: [A, B, C, E]
E: [B, D]
```

Aspect	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	O(V + E)
Edge lookup	O(1)	O(degree)
Best for	Dense graphs	Sparse graphs

7.4 Graph Operations

Insertion, Deletion, Traversal.

1. Insertion

(i) Add a node (vertex): `add_node("F")`

The new vertex `F` initially has no edges.

	A	B	C	D	E	F
A	0	1	1	1	0	0
B	1	0	0	1	1	0
C	1	0	0	1	0	0
D	1	1	1	0	1	0
E	0	1	0	1	0	0
F	0	0	0	0	0	0

```
A: [B, C, D]
B: [A, D, E]
C: [A, D]
D: [A, B, C, E]
E: [B, D]
F: []          <- new vertex
```

(ii) Add an edge: `add_edge("E", "F")`

```
graph[E][F] = 1
graph[F][E] = 1
```

Updated lists: `E: [B, D, F]` and `F: [E]`.

Insertion functions: adjacency matrix

```
nodes = []        # list of node names
graph = []        # matrix (list of lists)
node_count = 0

def add_node(v):
    global node_count
    if v in nodes:
        print(v, "is already present in the graph")
    else:
        node_count += 1
        nodes.append(v)
        for n in graph:              # add a new column to each existing row
            n.append(0)
        temp = []
        for i in range(node_count):  # add a new row
            temp.append(0)
        graph.append(temp)

def add_edge(v1, v2):
    if v1 not in nodes:
        print(v1, "is not present in the graph")
    elif v2 not in nodes:
        print(v2, "is not present in the graph")
    else:
        index1 = nodes.index(v1)
        index2 = nodes.index(v2)
        graph[index1][index2] = 1
        graph[index2][index1] = 1    # undirected graph

def print_graph():
    for i in range(node_count):
        for j in range(node_count):
            print(graph[i][j], end=" ")
        print()
```

```
print("Before adding nodes")
print(nodes)
print(graph)
add_node("A")
add_node("B")
print("After adding nodes")
print("nodes:", nodes)
print(graph)
print_graph()
```

Insertion functions: adjacency list

```
graph = {}

def add_node(v):
    if v in graph:
        print(v, "is already present in graph")
    else:
        graph[v] = []

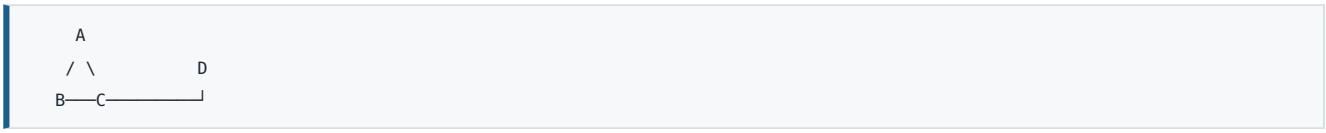
def add_edge(v1, v2):
    if v1 not in graph:
        print(v1, "is not present in the graph")
    elif v2 not in graph:
        print(v2, "is not present in the graph")
    else:
        graph[v1].append(v2)
        graph[v2].append(v1)         # undirected graph

add_node("A")
add_node("B")
add_edge("A", "B")
print(graph)     # {'A': ['B'], 'B': ['A']}
```

2. Deletion

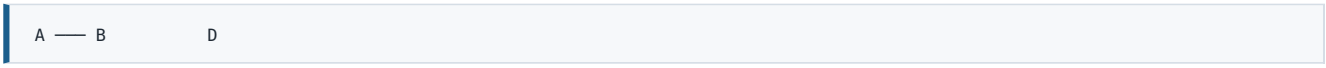
(i) Delete a node: `delete_node("C")`

Before (edges: A–B, A–C, B–C, C–D):




```
A: [B, C]      B: [A, C]      C: [A, B, D]      D: [C]
```

After deleting `C`, all edges touching `C` are removed:




```
A: [B]      B: [A]      D: []
```

	A	B	D
A	0	1	0
B	1	0	0
D	0	0	0

(ii) Delete an edge: `delete_edge("C", "D")`

Using the 5-node graph from Section 7.3, the edge C–D is removed.

```
A: [B, C, D]
B: [A, D, E]
C: [A]
D: [A, B, E]
E: [B, D]
```

In the matrix, the entries `[C][D]` and `[D][C]` become `0`.

Deletion functions: adjacency matrix

```
def delete_node(v):
    global node_count
    if v not in nodes:
        print(v, "is not present in the graph")
    else:
        index1 = nodes.index(v)
        node_count -= 1
        nodes.remove(v)
        graph.pop(index1)            # remove the row
        for row in graph:
            row.pop(index1)          # remove the column

def delete_edge(v1, v2):
    if v1 not in nodes:
        print(v1, "is not present in graph")
    elif v2 not in nodes:
        print(v2, "is not present in graph")
    else:
        index1 = nodes.index(v1)
        index2 = nodes.index(v2)
        graph[index1][index2] = 0
        graph[index2][index1] = 0
```

Deletion functions: adjacency list

```
def delete_node(v):
    if v not in graph:
        print(v, "is not present in the graph")
    else:
        graph.pop(v)
        for i in graph:
            list1 = graph[i]
            if v in list1:
                list1.remove(v)

def delete_edge(v1, v2):
    if v1 not in graph:
        print(v1, "is not present in the graph")
    elif v2 not in graph:
        print(v2, "is not present in the graph")
    else:
        if v2 in graph[v1]:
            graph[v1].remove(v2)
            graph[v2].remove(v1)
```

3. Traversal

Two strategies: **DFS** and **BFS**.

(1) DFS: Depth First Search

Visit the starting node first, then go as deep as possible before backtracking.

Steps:

1. Consider the starting node as the current node and visit it.
2. Visit an **unvisited adjacent node** of the current node and make it the current node.
3. Repeat step 2 until a dead end is reached.
4. If unvisited nodes remain in the graph, **backtrack**: take the most recently visited node as the current node and

repeat step 2.

Example traversal order from the notes' diagram: `A → B → C → D → K → M → F`

DFS: recursive implementation

The graph used below (build it with the adjacency-list `add_node` / `add_edge` above):

```
Edges: A–B, B–E, A–C, A–D, B–D, C–D, E–D
```

```
def DFS(node, visited, graph):
    if node not in graph:
        print("Node is not present")
        return
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for i in graph[node]:
            DFS(i, visited, graph)

visited = set()
graph = {}
for v in ["A", "B", "C", "D", "E"]:
    add_node(v)
for v1, v2 in [("A", "B"), ("B", "E"), ("A", "C"),
               ("A", "D"), ("B", "D"), ("C", "D"), ("E", "D")]:
    add_edge(v1, v2)

print(graph)
# {'A': ['B', 'C', 'D'], 'B': ['A', 'E', 'D'], 'C': ['A', 'D'],
#  'D': ['A', 'B', 'C', 'E'], 'E': ['B', 'D']}

DFS("A", visited, graph)    # A B E D C
```

> **Note (correction):** The handwritten version printed "Node is not present" but then kept executing, which would raise a `KeyError` on `graph[node]`. A `return` was added.

DFS: iterative implementation (explicit stack)

```
def DFSiterative(node, graph):
    visited = set()
    if node not in graph:
        print("Node is not present")
        return
    stack = []
    stack.append(node)
    while stack:
        current = stack.pop()
        if current not in visited:
            print(current, end=" ")
            visited.add(current)
            for i in graph[current]:
                stack.append(i)

graph = {}
for v in ["A", "B", "C", "D", "E"]:
    add_node(v)
for v1, v2 in [("A", "B"), ("B", "E"), ("A", "C"),
               ("A", "D"), ("B", "D"), ("C", "D"), ("E", "D")]:
    add_edge(v1, v2)

print(graph)
DFSiterative("A", graph)    # A D E B C
```

> The iterative version visits neighbours in **reverse** order compared with the recursive one, because the stack pops the most recently pushed neighbour first. Both outputs are valid DFS orders.

Complexity: DFS runs in O(V + E) with an adjacency list and O(V²) with an adjacency matrix.

(2) BFS: Breadth First Search (supplementary)

The handwritten notes list BFS under traversal but do not detail it. This short reference completes the set. BFS visits nodes level by level using a **queue**.

```
from collections import deque

def BFS(start, graph):
    if start not in graph:
        print("Node is not present")
        return
    visited = {start}
    q = deque([start])
    while q:
        current = q.popleft()
        print(current, end=" ")
        for neighbour in graph[current]:
            if neighbour not in visited:
                visited.add(neighbour)
                q.append(neighbour)

BFS("A", graph)    # A B C D E
```

Quick Reference Summary

Structure	Ordering	Mutable	Key idea	Typical Python tool
List	Ordered	Yes	Dynamic array	`list`
Tuple	Ordered	No	Fast, read-only	`tuple`
Set	Unordered	Yes	Unique elements	`set`
Dictionary	Insertion-ordered (3.7+)	Yes	Key → value	`dict`
Stack	LIFO	n/a	push/pop at top	`list`, `deque`, `LifoQueue`
Queue	FIFO	n/a	enqueue rear, dequeue front	`deque`, `queue.Queue`
Priority Queue	By priority	n/a	Smallest (or largest) first	`heapq`, `PriorityQueue`
Linked List	Sequential	n/a	Nodes joined by references	custom `Node` class
BST	Sorted (in-order)	n/a	left < node < right	custom `BST` class
Binary Heap	Heap property	n/a	Complete tree in an array	`heapq`
Graph	None	n/a	Vertices + edges	adjacency matrix or list